



组合数据类型概述



引入问题：掷色子

- **任务**：将一颗色子掷 6000 次，统计每种点数出现的次数
- **思路**：用 `random.randint(1, 6)` 模拟掷一次色子，用 6 个变量分别记录 1~6 点出现的次数

引入问题：掷色子

```
import random
```

```
f1 = f2 = f3 = f4 = f5 = f6 = 0
for i in range(6000):
    face = random.randint(1, 6)
    if face == 1:
        f1 += 1
    elif face == 2:
        f2 += 1
    elif face == 3:
        f3 += 1
    elif face == 4:
        f4 += 1
    elif face == 5:
        f5 += 1
    else:
        f6 += 1
print("1 点出现了", f1, "次")
print("2 点出现了", f2, "次")
print("3 点出现了", f3, "次")
print("4 点出现了", f4, "次")
print("5 点出现了", f5, "次")
print("6 点出现了", f6, "次")
```

运行结果（每次运行结果随机）：

```
1 点出现了 989 次
2 点出现了 1009 次
3 点出现了 998 次
4 点出现了 991 次
5 点出现了 950 次
6 点出现了 1063 次
```

这段代码能完成任务，
但写得好吗？

引入问题：掷色子

■ 上一页代码存在的问题：

- **变量多**：6 种点数就要定义 6 个变量
- **分支多**：每种点数都要写一个 if 分支，结构完全相同
- **输出重复**：6 行 print 只有数字不同

■ 如果掷两颗色子呢？

- 两颗色子的点数之和是 2~12，共 11 种结果：

```
python
```

```
f2 = f3 = f4 = f5 = f6 = f7 = f8 = f9 = f10 = f11 = f12 = 0
```

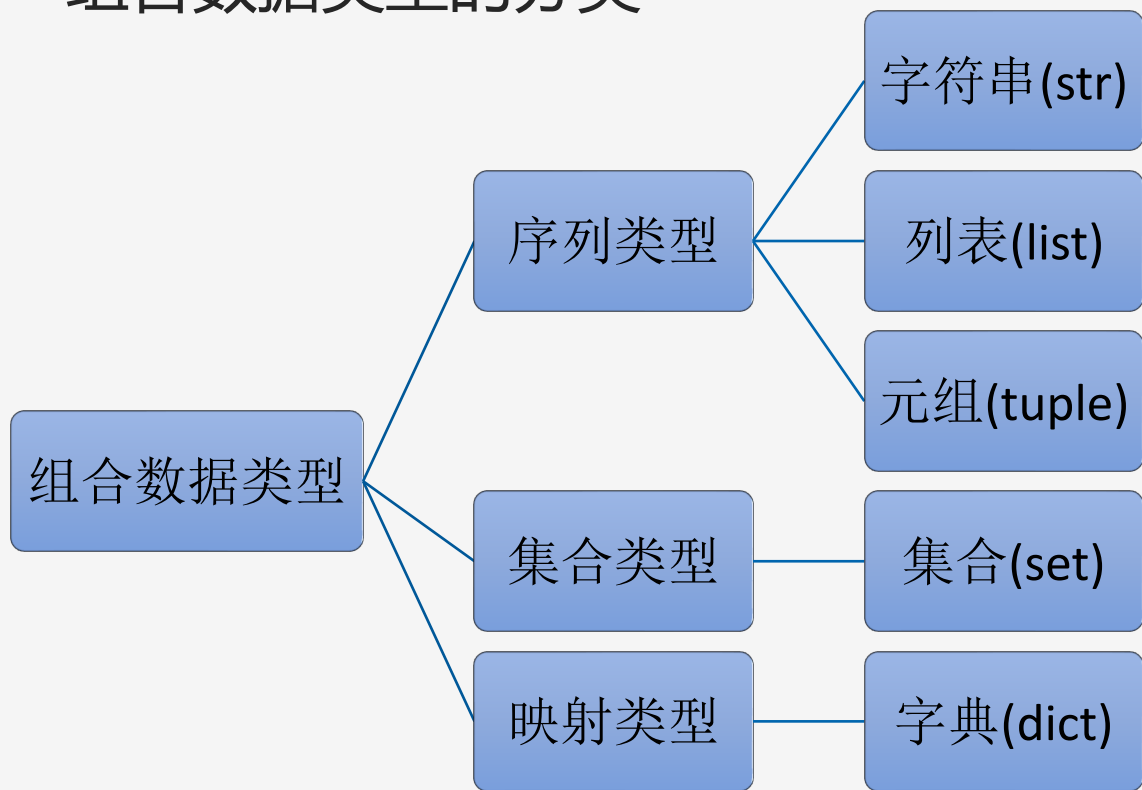
需要 11 个变量、11 个分支、11 行输出。如果掷三颗、四颗，代码量会继续增加

引入问题：掷色子

■ 思考：

- 能否用**一个变量**保存多个数据？
- 能否用**统一的代码**对多个数据进行操作？

■ 组合数据类型的分类



序列类型：元素有先后顺序，通过序号访问

集合类型：元素无序，不能重复

映射类型：元素是“键-值”对，通过键访问

可变类型与不可变类型

■ 概念：

- **可变类型**：创建后，可以添加、删除或修改其中的元素
- **不可变类型**：创建后，其中的元素不能添加、删除或修改

类型	是否可变
字符串 (str)	不可变
元组 (tuple)	不可变
列表 (list)	可变
集合 (set)	可变
字典 (dict)	可变

可变类型与不可变类型

■ 概念：

- **可变类型**：创建后，可以添加、删除或修改其中的元素
- **不可变类型**：创建后，其中的元素不能添加、删除或修改

■ 以字符串为例：

```
s = "hello"
```

```
print(s[0])
```

```
s[0] = "H"
```

```
h
```

```
Traceback (most recent call last):
```

```
File "test.py", line 3, in <module>
```

```
s[0] = "H"
```

```
TypeError: 'str' object does not support item assignment
```

报错，可以读取字符串中的字符，但不能修改，说明字符串是不可变类型

不支持元素赋值



列表 (List)

1. 什么是列表
2. 创建列表
3. 列表的运算
4. 遍历列表
5. 列表的方法
6. 列表生成式
7. 嵌套列表
8. 列表的拷贝
9. 列表的应用



列表 (List)

■ 什么是列表？

- 列表是由一系列**元素按特定顺序**构成的数据序列，一个列表变量可以保存多个数据

■ 列表的特点

特点	说明
有序	元素按放入的顺序排列，每个元素有自己的位置
可变	可以添加、删除、修改元素
可重复	同一个值可以出现多次
类型不限	元素可以是不同类型的数据
长度不限	元素个数没有限制，可以随时增减

列表 (List)

■ 列表示意图

之前：6 个独立的变量



现在：1 个列表



列表 (List)

■ 创建列表

- 用方括号 `[]` 把元素括起来，元素之间用逗号分隔
- 语法：列表名 = [元素1, 元素2, ..., 元素n]

`nums = [35, 12, 99, 68, 55, 35, 87]` 列表**允许重复元素**

`languages = ['Python', 'Java', 'Go', 'Kotlin']`

`items = [100, 12.3, 'Python', True]` 列表**允许不同类型的元素**

`empty = []` **空列表**，常用于先创建、再逐个添加元素

```
print(nums)
```

```
print(languages)
```

```
print(items)
```

```
print(empty)
```

```
print(type(nums))
```

运行结果：

```
[35, 12, 99, 68, 55, 35, 87]
['Python', 'Java', 'Go', 'Kotlin']
[100, 12.3, 'Python', True]
[]
<class 'list'>
```



列表 (List)

■ 创建列表

- 用 `list()` 把其他序列转换成列表
- 语法：列表名 = `list(序列)`

```
nums = list(range(1, 10))
```

```
letters = list('hello')
```

```
empty = list()
```

```
print(nums)
```

```
print(letters)
```

```
print(empty)
```

运行结果：

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
['h', 'e', 'l', 'l', 'o']
```

```
[]
```

`range(1, 10)` 产生 1~9 的整数序列，`list()` 将它们变成列表的元素

字符串是由字符构成的序列，`list('hello')` 把每个字符变成一个元素，重复的 l 保留两个

`list()` 不传参数时，得到空列表，与 `[]` 作用相同



列表 (List)

■ 列表的运算：拼接和重复

运算符	作用	示例
+	拼接：将两个列表的元素连接成一个新列表	<code>[1, 2] + [3] → [1, 2, 3]</code>
+=	拼接后赋值给原变量	<code>a += b</code> 相当于 <code>a = a + b</code>
*	重复：将列表元素重复指定次数	<code>[1, 2] * 2 → [1, 2, 1, 2]</code>

```
nums1 = [35, 12, 99]
nums2 = [45, 58]
languages = ['Python', 'Java']
print(nums1 + nums2)
print(nums2 + languages)
nums1 += nums2
print(nums1)
print(nums2 * 3)
print(languages * 2)
counters = [0] * 6
print(counters)
```

运行结果：

```
[35, 12, 99, 45, 58]
[45, 58, 'Python', 'Java']
[35, 12, 99, 45, 58]
[45, 58, 45, 58, 45, 58]
['Python', 'Java', 'Python', 'Java']
[0, 0, 0, 0, 0, 0]
```

`[0] * 6` 可以快速创建包含 6 个 0 的列表，
常用于初始化计数器



列表 (List)

■ 列表的运算：成员运算

运算符	作用
<code>x in 列表</code>	x 在列表中，返回 True，否则返回 False
<code>x not in 列表</code>	x 不在列表中，返回 True，否则返回 False

```
nums = [45, 58, 29]
languages = ['Python', 'Java', 'JavaScript']
print(29 in nums)
print(99 in nums)
print('C++' not in languages)
print('Python' not in languages)
print('java' in languages)
```

判断时要求元素完全相同：'java' 与 'Java'
大小写不同，所以结果为 False

运行结果：

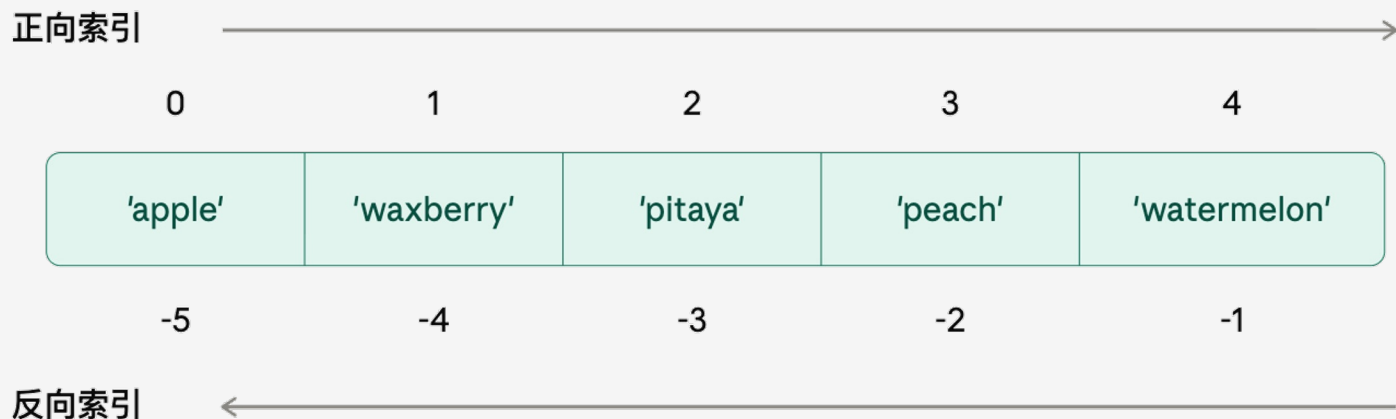
```
True
False
True
False
False
```



列表 (List)

■ 列表的运算：索引运算

- 语法：**列表名[索引]**，通过元素的位置编号访问元素



索引方式	范围	第一个元素	最后一个元素
正向索引	0 ~ N-1	[0]	[N-1]
反向索引	-1 ~ -N	[-N]	[-1]



列表 (List)

■ 列表的运算：索引运算

- 如果索引超出了有效范围，会发生什么？

```
fruits = ['apple', 'waxberry', 'pitaya', 'peach', 'watermelon']  
print(len(fruits))  
print(fruits[5])
```

运行结果：

```
5  
Traceback (most recent call last):  
  File "test.py", line 3, in <module>  
    print(fruits[5])  
IndexError: list index out of range
```



列表 (List)

■ 列表的运算：切片运算

- 语法：列表名[start:end:stride]

参数	含义
start	起始位置（包含）
end	终止位置（不包含）
stride	步长，即每次位置的增量



列表 (List)

■ 列表的运算：切片运算

- 语法：列表名[start:end:stride]

```
fruits = ['apple', 'strawberry', 'durian', 'peach', 'watermelon']  
print(fruits[1:3:1])  
print(fruits[0:3:1])  
print(fruits[0:5:2])  
print(fruits[-4:-2:1])
```

0	1	2	3	4
'apple'	'strawberry'	'durian'	'peach'	'watermelon'

运行结果：

```
['strawberry', 'durian']  
['apple', 'strawberry', 'durian']  
['apple', 'durian', 'watermelon']  
['strawberry', 'durian']
```



列表 (List)

- 列表的运算：切片运算
 - 切片的省略写法

省略规则：

参数	可省略的情况	省略后的默认值
start	从头开始	步长为正时为 0
end	取到末尾	步长为正时为 N
stride	步长为 1	1



列表 (List)

■ 列表的运算：切片运算

• 切片的省略写法

```
fruits = ['apple', 'strawberry', 'durian', 'peach', 'watermelon']  
print(fruits[1:3])      # 省略 stride, 等价于 fruits[1:3:1]  
print(fruits[:3])       # 省略 start 和 stride, 等价于 fruits[0:3:1]  
print(fruits[::2])      # 省略 start 和 end, 等价于 fruits[0:5:2]  
print(fruits[2:])       # 省略 end 和 stride, 等价于 fruits[2:5:1]
```

运行结果：

```
['strawberry', 'durian']  
['apple', 'strawberry', 'durian']  
['apple', 'durian', 'watermelon']  
['durian', 'peach', 'watermelon']
```



列表 (List)

■ 列表的运算：切片运算

- 步长为负数时，从右向左取元素，此时 start 应在 end 的右边
- `[::-1]` 可以得到倒序的新列表

```
fruits = ['apple', 'strawberry', 'durian', 'peach', 'watermelon']  
print(fruits[-2:-6:-1])  
print(fruits[-2::-1]) 等价，省略 end，取到第一个元素  
print(fruits[::-1])
```

运行结果：

```
['peach', 'durian', 'strawberry', 'apple']  
['peach', 'durian', 'strawberry', 'apple']  
['watermelon', 'peach', 'durian', 'strawberry', 'apple']
```



列表 (List)

■ 列表的运算：切片运算

- ⚠ 当步长为负数时，若 start 在 end 左侧，结果是空列表，不会报错

```
fruits = ['apple', 'strawberry', 'durian', 'peach', 'watermelon']  
print(fruits[1:3:-1])      # 步长为负，start 在 end 左侧  
print(fruits[-4:-2:-1])    # 同上，使用反向索引
```

均返回空列表 []，步长为正而 start 在 end 右侧时同理



列表 (List)

■ 列表的运算：切片运算

-  当步长为负数时，若 start 在 end 左侧，结果是空列表，不会报错

步长	start 与 end 的位置关系	结果
正数（向右取）	start 在 end 左侧	正常取值
正数（向右取）	start 在 end 右侧或相同	空列表
负数（向左取）	start 在 end 右侧	正常取值
负数（向左取）	start 在 end 左侧或相同	空列表

步长的方向必须是从 start 走向 end，否则得到空列表



列表 (List)

■ 列表的运算：切片运算

- **⚠️ 切片不会越界报错**，而是取到末尾为止，这一点与索引不同

```
fruits = ['apple', 'strawberry', 'durian', 'peach', 'watermelon']
print(fruits[1:100])      # end 超出范围，取到最后一个元素为止
print(fruits[-100:2])     # start 超出范围，从第一个元素开始
print(fruits[10:20])      # start 和 end 都超出范围，得到空列表
print(fruits[100])        # 索引超出范围，报错
```

运行结果：

```
['strawberry', 'durian', 'peach', 'watermelon']
['apple', 'strawberry']
[]
Traceback (most recent call last):
  File "test.py", line 5, in <module>
    print(fruits[100])
IndexError: list index out of range
```

- 切片的 start 或 end 超出范围时，Python 会自动将其限制在列表的有效范围内，不会报错
- 整个范围都在列表之外时，得到空列表
- 索引访问的是一个具体位置，位置不存在就会报错

列表 (List)

■ 列表的运算：切片运算

- 通过切片修改元素

- **语法**：列表名[start:end] = 新列表

- **作用**：用新列表整体**替换**切片范围内的元素，新列表的长度**不要求**与切片范围的长度相同

```
nums = [0, 1, 2, 3, 4]
nums[1:3] = ['a', 'b']
print(nums)
```

```
nums = [0, 1, 2, 3, 4]
nums[1:3] = ['a', 'b', 'c', 'd']
print(nums)
```

```
nums = [0, 1, 2, 3, 4]
nums[1:3] = ['a']
print(nums)
```

```
nums = [0, 1, 2, 3, 4]
nums[1:3] = []
print(nums)
```





列表 (List)

■ 列表的运算：切片运算

• 通过切片修改元素

- **语法**：列表名[start:end] = 新列表

- **作用**：用新列表整体**替换**切片范围内的元素，新列表的长度**不要求**与切片范围的长度相同

```
nums = [0, 1, 2, 3, 4]
```

```
nums[1:3] = ['a', 'b']
```

```
print(nums)
```



运行结果：

```
[0, 'a', 'b', 3, 4]
```

```
[0, 'a', 'b', 'c', 'd', 3, 4]
```

```
nums = [0, 1, 2, 3, 4]
```

```
nums[1:3] = ['a', 'b', 'c', 'd']
```

```
print(nums)
```

新列表	元素个数	效果	列表长度变化
['a', 'b']	2个, 相等	一一替换	5→5
['a', 'b', 'c', 'd']	4个, 多2个	替换后多出的插入	5→7



列表 (List)

■ 列表的运算：切片运算

• 通过切片修改元素

- **语法**：列表名[start:end] = 新列表

- **作用**：用新列表整体**替换**切片范围内的元素，新列表的长度**不要求**与切片范围的长度相同

```
nums = [0, 1, 2, 3, 4]
```

```
nums[1:3] = ['a']
```

```
print(nums)
```

```
nums = [0, 1, 2, 3, 4]
```

```
nums[1:3] = []
```

```
print(nums)
```

运行结果：

[0, 'a', 3, 4]

[0, 3, 4]



['a']	1个，少1个	替换后缺少的位置删除	5 → 4
[]	0个	切片范围内的元素全部删除	5 → 3

列表 (List)

■ 列表的运算：比较运算

- 运算符：`==`、`!=`、`<`、`<=`、`>`、`>=`
- 比较规则：从第一个元素开始**逐个比较**对应位置的元素
 - 遇到第一对不相等的元素，由这对元素的大小决定结果，**后面的元素不再比较**
 - 如果一个列表的元素比完了仍全部相等，**元素少的列表更小**

```
nums1 = [1, 2, 3, 4]
nums2 = list(range(1, 5))
nums3 = [3, 2, 1]
print(nums1 == nums2)
print(nums1 != nums2)
print(nums1 < nums3)
print([1, 2] < [1, 2, 3])
print(['apple', 'peach'] < ['banana'])
```



列表 (List)

■ 列表的运算：比较运算

- 运算符：`==`、`!=`、`<`、`<=`、`>`、`>=`
- 比较规则：从第一个元素开始**逐个比较**对应位置的元素
 - 遇到第一对不相等的元素，由这对元素的大小决定结果，**后面的元素不再比较**
 - 如果一个列表的元素比完了仍全部相等，**元素少的列表更小**

```
nums1 = [1, 2, 3, 4]
nums2 = list(range(1, 5))
nums3 = [3, 2, 1]
print(nums1 == nums2)
print(nums1 != nums2)
print(nums1 < nums3)
print([1, 2] < [1, 2, 3])
print(['apple', 'peach'] < ['banana'])
```

运行结果：



```
True
False
True
True
True
```



列表 (List)

■ 列表的运算：比较运算

```
nums1 = [1, 2, 3, 4]
nums2 = list(range(1, 5))
nums3 = [3, 2, 1]
```

表达式	比较过程	结果
<code>nums1 == nums2</code>	两个列表都是 <code>[1, 2, 3, 4]</code> ，对应元素全部相同	True
<code>nums1 != nums2</code>	同上	False
<code>nums1 < nums3</code>	第一对元素 $1 < 3$ ，直接得出结果，后面不再比较	True
<code>[1, 2] < [1, 2, 3]</code>	前两对相等，左边元素比完，元素少的更小	True
<code>['apple', 'peach'] < ['banana']</code>	第一对 'apple' < 'banana'（按字母顺序）	True

⚠ 对应位置的元素必须能够比较大小，否则报错，`print([1, 2] < ['a', 'b'])` 会报错，**`TypeError: '<' not supported between instances of 'int' and 'str'`**

列表 (List)

- 列表的运算：for 循环遍历列表中每一个元素

方法一：通过索引遍历

python

```
languages = ['Python', 'Java', 'C++', 'Kotlin']  
for i in range(len(languages)):  
    print(i, languages[i])
```

运行结果：

```
0 Python  
1 Java  
2 C++  
3 Kotlin
```

len(languages) 为 4，range(4) 产生 0 ~ 3，正好是列表的全部有效索引

列表 (List)

- 列表的运算：for 循环遍历列表中每一个元素

方法二：直接遍历元素

python

```
languages = ['Python', 'Java', 'C++', 'Kotlin']  
for language in languages:  
    print(language)
```

运行结果：

```
0 Python  
1 Java  
2 C++  
3 Kotlin
```

每次循环，变量 language 依次代表列表中的一个元素



列表 (List)

- 列表的运算：for 循环遍历列表中每一个元素
 - 两种方法的比较：

	方法一：通过索引	方法二：直接遍历
循环变量代表	索引（位置）	元素本身
能否得到元素位置	能	不能
能否修改列表元素	能， <code>languages[i] = 新值</code>	不能
代码简洁程度	较繁琐	简洁
适用场景	需要用到位置，或需要修改元素	只需要读取元素



列表 (List)

■ 用列表重构掷色子程序

- **回顾问题**：将一颗色子掷 6000 次，统计每种点数出现的次数

```
import random
```

```
counters = [0] * 6                                # 创建计数器列表，6 个元素分别记录 1~6 点的次数
for i in range(6000):                              # 掷 6000 次色子
    face = random.randint(1, 6)                    # 模拟掷一次，得到 1~6 的点数
    counters[face - 1] += 1                         # 点数 face 对应索引 face-1，该位置次数加 1
for face in range(1, 7):                           # 依次输出 1~6 点的统计结果
    print(face, "点出现了", counters[face - 1], "次")
```





列表 (List)

■ 用列表重构掷色子程序

• 拓展问题：掷两颗色子

```
import random
```

```
counters = [0] * 11          # 两颗色子点数之和为 2~12, 共 11 种结果
for i in range(6000):        # 掷 6000 次
    total = random.randint(1, 6) + random.randint(1, 6)  # 两颗色子点数相加
    counters[total - 2] += 1  # 点数和 total 对应索引 total-2, 该位置次数加 1
for total in range(2, 13):   # 依次输出 2~12 点的统计结果
    print(total, "点出现了", counters[total - 2], "次")
```

